

EcoSense

환경 모니터링 IoT 대시보드

소프트웨어 설계 문서 (Design Document)

17조

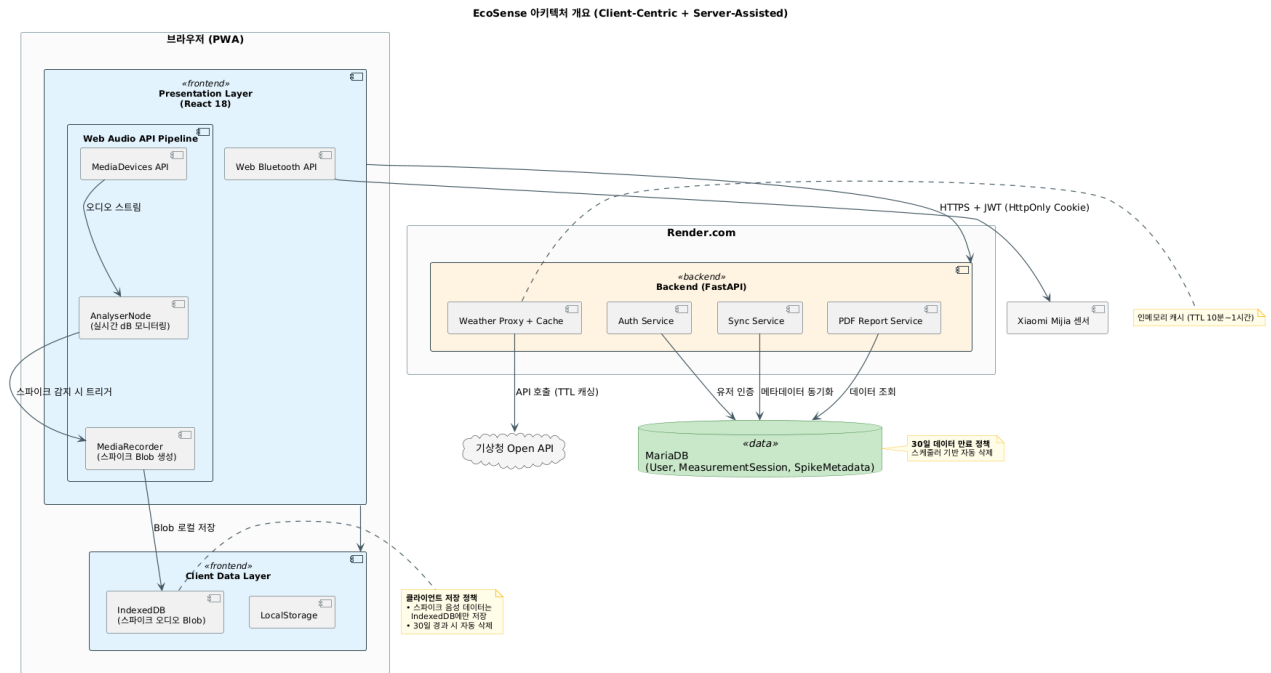
정우진, 이승호, 선지수, 정원제

2026년 5월

1. 아키텍처 개요

1.1 전체 시스템 아키텍처

EcoSense는 클라이언트 중심(Client-Centric) + 서버 보조(Server-Assisted) 구조로 설계한다. 실시간 오디오 처리와 센서 제어는 브라우저에서 직접 수행하며, 백엔드는 인증, 데이터 메타데이터 영속화, PDF 생성, 외부 API 프록시 역할을 담당한다.



[그림 1] EcoSense 전체 시스템 아키텍처

1.2 레이어 상세 설명

Presentation Layer (Frontend)

- 기술 스택: React 18 + Vite + Tailwind CSS + Chart.js
- 배포: Vercel
- Web Audio API Pipeline: AnalyserNode(실시간 dB 모니터링), MediaRecorder(스파이크 Blob 버퍼링)
- Web Bluetooth API를 통한 Xiaomi Mijia 센서 데이터 수신

Client Data Layer

- IndexedDB에 수면 스파이크 음성 데이터(Blob)를 직접 저장. 로그인 사용자도 Blob은 클라이언트에 보관하고, 서버에는 메타데이터만 동기화. 30일 경과 데이터 자동 삭제 로직 적용.

Business Logic Layer (Backend)

- 기술 스택: FastAPI (Python), 배포: Render.com. 인증, 메타데이터 동기화, PDF 생성, 기상청 API 프록시 + TTL 인메모리 캐싱 담당.

Data Layer

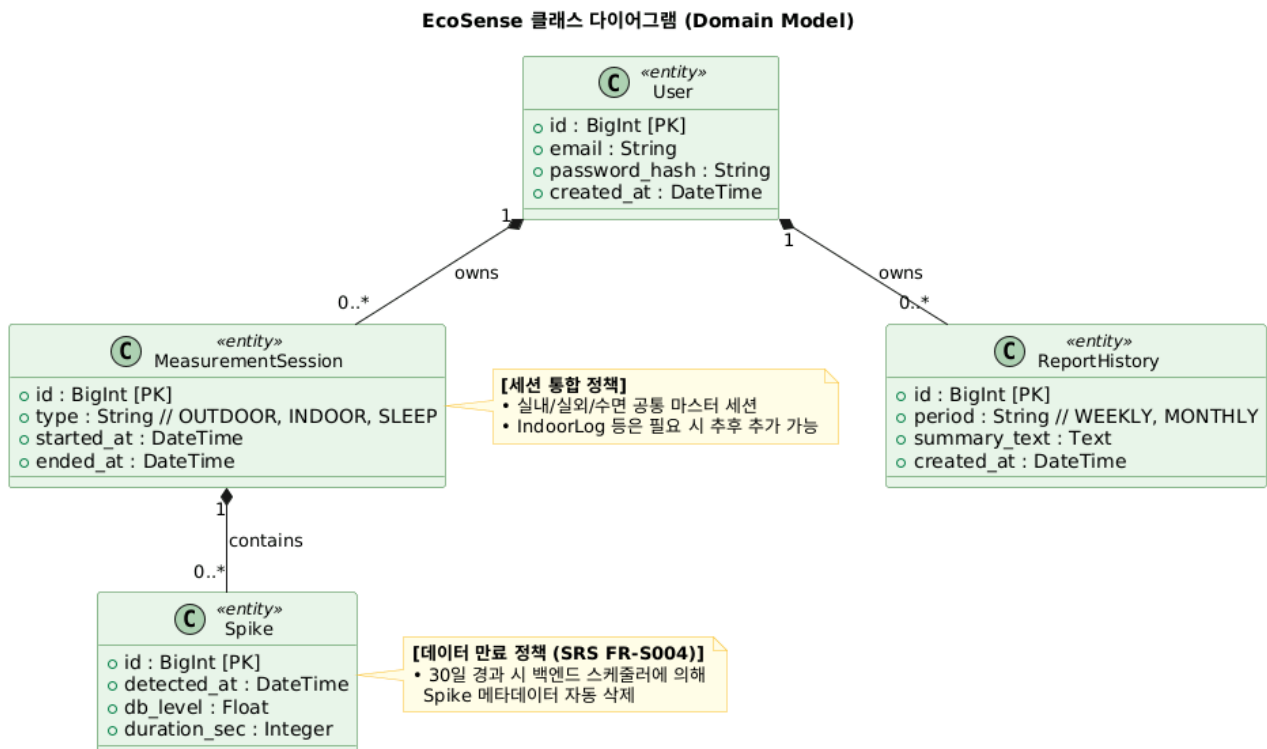
- MariaDB (Render): 로그인 사용자의 영속 데이터 저장. 주요 엔티티: User, MeasurementSession, Spike. 스파이크 음성 오디오 파일 자체는 저장하지 않음.

1.3 인증 토큰 관리

JWT는 HttpOnly Secure Cookie 방식으로 관리하여 XSS 공격 위험을 최소화한다. LocalStorage 저장 방식은 보안상 지양한다.

2. 클래스 다이어그램

EcoSense의 도메인 모델은 User, MeasurementSession, Spike, ReportHistory 4개의 핵심 엔티티로 구성된다. MeasurementSession은 실내/실외/수면을 type으로 구분하며, Spike는 수면 중 감지된 비정상 소음 메타데이터를 저장한다. ReportHistory는 PDF 보고서 생성 이력을 관리한다.



[그림 2] EcoSense 클래스 다이어그램 (Domain Model)

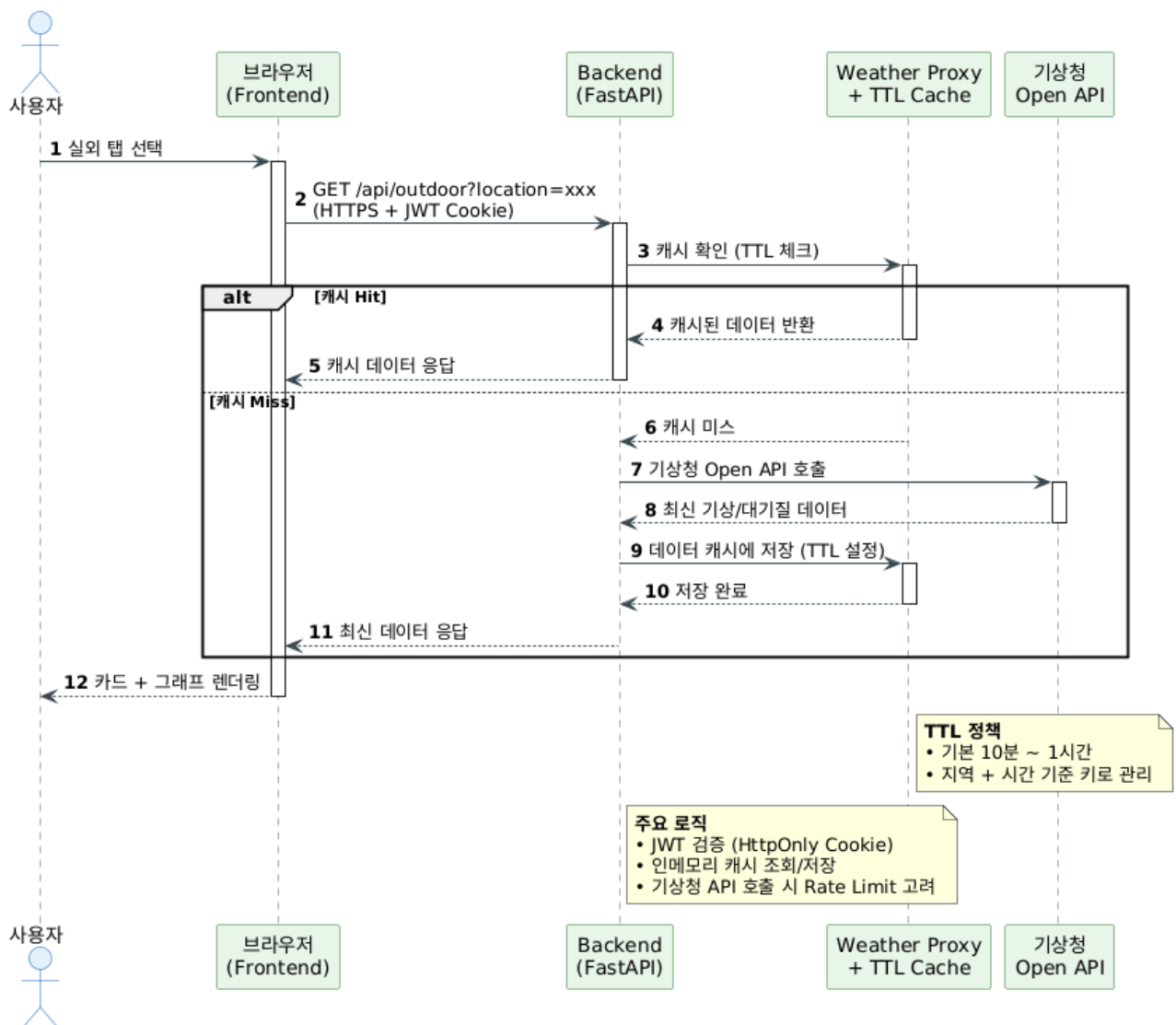
3. 시퀀스 다이어그램

주요 4개의 유스케이스에 대해 시퀀스 다이어그램을 작성하였다. 각 다이어그램은 시스템의 핵심 동작 흐름과 아키텍처 특성을 잘 보여준다.

3.1 UC-001 실외 대시보드 조회

기상청 Open API 호출 시 TTL 기반 인메모리 캐싱을 적용하여 불필요한 API 호출을 줄이고 응답 속도를 개선하였다.

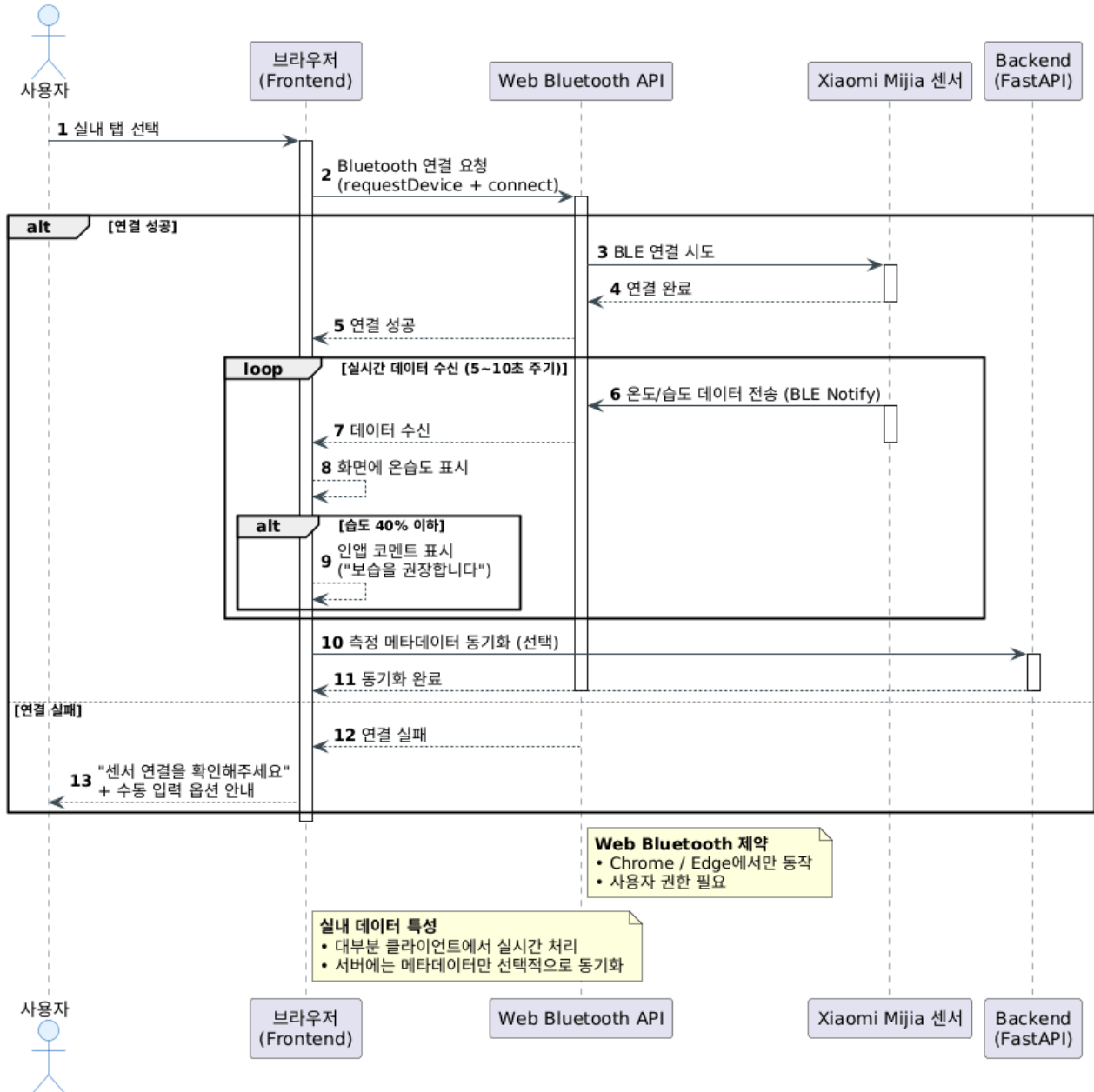
UC-001 실외 대시보드 조회 시퀀스 다이어그램



3.2 UC-002 실내 대시보드 조회

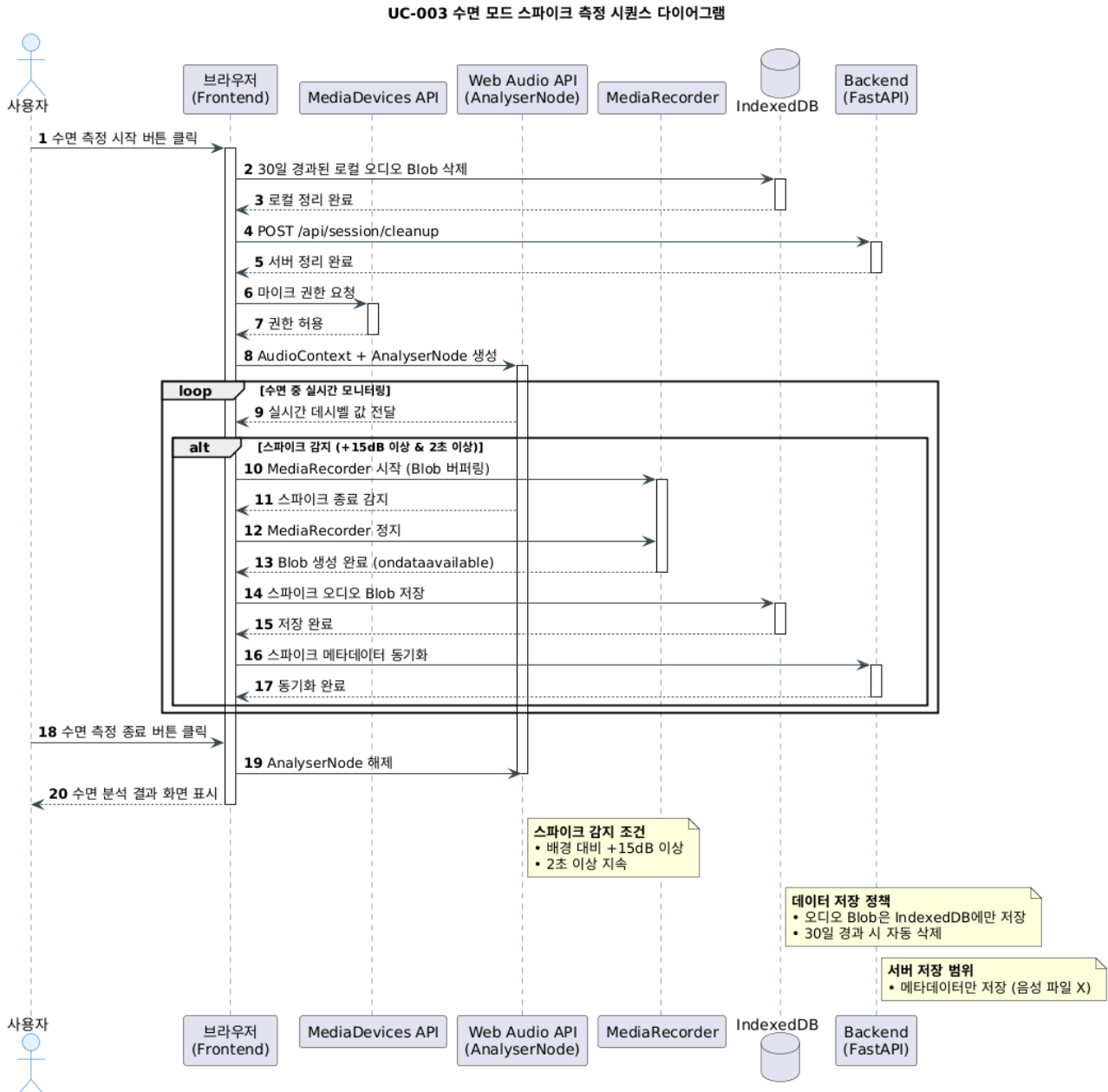
Web Bluetooth API를 통해 Xiaomi Mijia 센서와 직접 통신하며, 실시간 온습도 데이터를 수신한다. 연결 실패 시 대체 UX를 제공한다.

UC-002 실내 대시보드 조회 시퀀스 다이어그램



3.3 UC-003 수면 모드 스파이크 측정

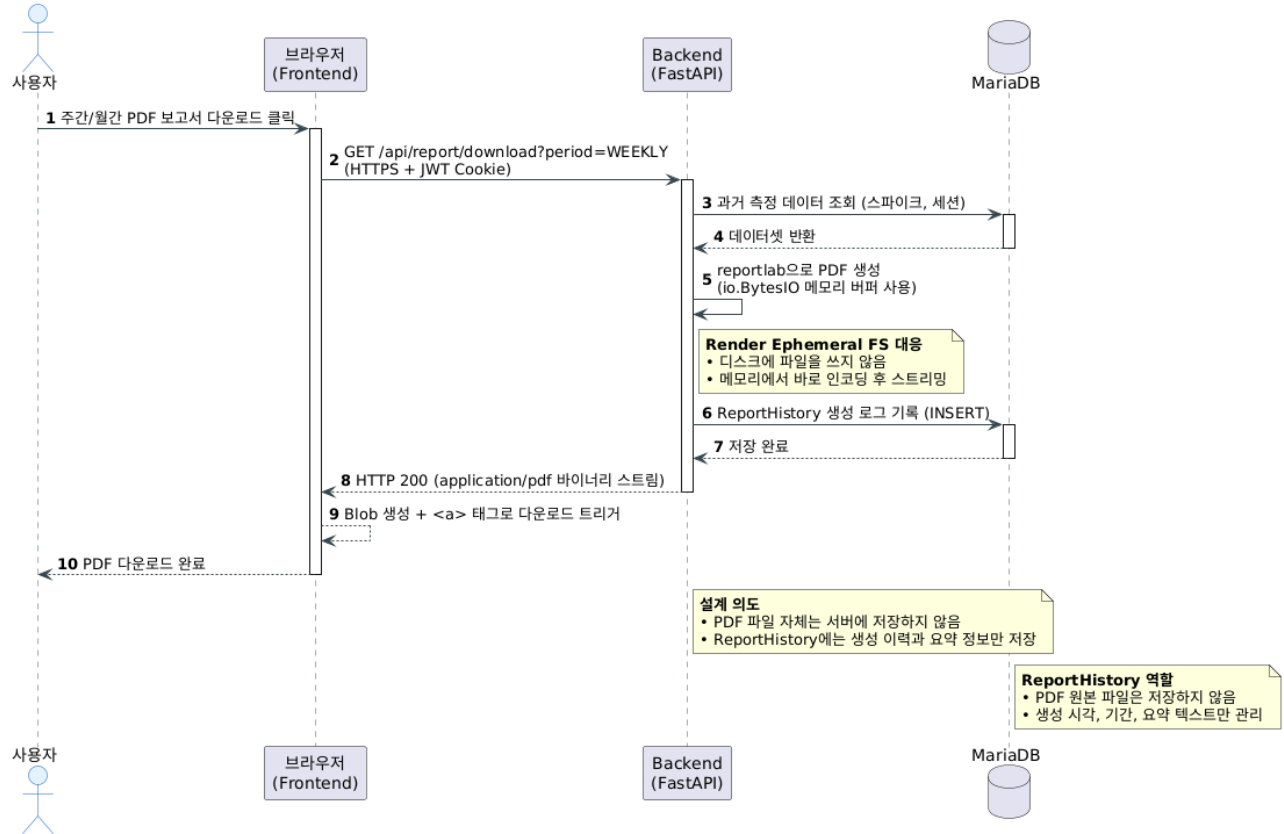
Web Audio API(AnalyserNode + MediaRecorder)를 사용하여 실시간 데시벨 모니터링 및 스파이크 감지 시 오디오 Blob을 생성한다. 생성된 Blob은 IndexedDB에 로컬 저장되며, 메타데이터만 서버로 동기화된다.



3.4 UC-004 PDF 보고서 생성

온디맨드 방식으로 PDF를 생성하며, Render.com의 Ephemeral File System 제약을 고려하여 서버 디스크에 파일을 저장하지 않고 메모리(io.BytesIO)에서 바로 인코딩하여 스트리밍한다.

UC-004 PDF 보고서 온디맨드 생성 시퀀스 다이어그램



4. 설계 결정 근거

4.1 클라이언트 중심 아키텍처 선택

Web Audio API와 Web Bluetooth API의 특성상 실시간 오디오 처리와 BLE 통신은 브라우저에서 직접 수행하는 것이 가장 효율적이다. 서버로 모든 데이터를 전송할 경우 지연 시간이 증가하고, Render 무료 플랜의 자원 제한을 초과할 위험이 있다. 따라서 Presentation Layer와 Client Data Layer에서 대부분의 처리를 담당하고, Backend는 메타데이터 동기화와 인증, PDF 생성 같은 보조 역할만 수행하도록 설계하였다.

4.2 데이터 저장 전략 (IndexedDB vs MariaDB)

수면 중 생성되는 스파이크 오디오 Blob은 용량이 크고 민감한 개인정보에 해당할 수 있다. 이를 서버에 저장할 경우 Render의 Ephemeral File System으로 인해 파일이 유실될 위험이 있으며, 트래픽 비용과 저장 비용도 증가한다. 따라서 오디오 Blob은 클라이언트의 IndexedDB에만 저장하

고, 서버(MariaDB)에는 감지 시각, 데시벨 수치, 지속 시간 등의 메타데이터만 저장하는 이원화 전략을 채택하였다. 이를 통해 개인정보 보호와 서버 부하를 동시에 해결하였다.

4.3 PDF 보고서 생성 방식

Render.com 무료 플랜은 서버가 재부팅되면 로컬 파일 시스템이 초기화되는 Ephemeral 구조이다. 따라서 PDF 파일을 서버 디스크에 저장하는 방식은 위험하다. 이에 따라 reportlab을 사용하여 메모리(io.BytesIO)에서 PDF를 실시간으로 생성하고, 생성 즉시 바이너리 스트림으로 클라이언트에 전달하는 On-demand 방식을 채택하였다. ReportHistory 테이블에는 생성 이력과 요약 정보만 저장하여 데이터 정합성을 유지한다.

4.4 30 일 데이터 만료 정책 구현

SRS FR-S004에 따라 측정 데이터는 30일 경과 시 자동 삭제되어야 한다. 수면 모드 진입 시점에 IndexedDB의 오래된 Blob과 MariaDB의 오래된 Spike 메타데이터를 동시에 정리하도록 하여, 사용자가 별도의 정리 작업을 하지 않아도 데이터가 자동으로 관리되도록 설계하였다.

4.5 기술 스택 및 인프라 선택

- Frontend: React + Vercel — PWA 지원과 빠른 정적 배포에 유리
- Backend: FastAPI (Python) — 비동기 처리 성능과 reportlab과의 호환성
- Database: MariaDB on Render — 관계형 데이터 무결성과 팀의 기존 경험
- 인증: HttpOnly Secure Cookie 기반 JWT — XSS 방어